

Final Project EE 267

Identifying a Dog's Weight-Class

03/14/2024

JUSTIN OGLE

PROFESSOR CHOO

MSEE, SPECIALIZATION ROBOTICS AND AUTONOMY

Table of Contents

1. Introduction.....	2
2. Description of Work.....	3
A. Design Process.....	3
B. Block Diagrams.....	7
C. Theory:.....	11
3. Simulation/Verification	13
A. Verification Results.....	13
B. Discussion	15
4. Conclusion.....	16
A. Project Summary.....	16
B. Goal Fulfillment.....	16
C. Future Work.....	16
5. References	18

1. Introduction

Identifying Dogs by their weight will aim to recognize man's best friend and future health problems their buddy might have in the future. This project addresses how an efficient and low-power convolutional neural network (CNN) is used by demonstrating its performance while comparing other performances with pretrained networks. I will do this by sorting various pictures of dogs into their weight classes in an organized folder structure and run various python programs to see what is the best fit.

The importances of this is at the pound, at the park, and around the neighborhood, it is possible to see a doggy in the window. Even though puppies are small they will grow into their full size which can be a problem of the jealous not-a-dog-owner. I want to provide a CNN or Pretrained Network that will identify the dog breed with their weight class defined to avoid some dog gone troubles just by using their phone to take a picture.

2. Description of Work

A. Design Process

First, I gathered a multi-dog breed database from Kaggle titled, "*Introducing DCGAN Dogs Images*". Next, I sort each breed into their individual weight classes while focusing on the heaviest that each canine can get to. Afterward, I would form the basic code templet to swap out the CNN and other Pretrained Networks.

The dogs are sorted into these weight-classes:

- Small Dog Breeds: 2 to 22 lbs.
- Medium Breeds: 24 to 57 lbs.
- Large Dog Breeds: 59 to 99 lbs.
- Giant or Extra-Large: 100lbs. or more

The Small Dogs Breeds into more sub classes as:

- Teacup: 4 lbs. or less
- Miniature: 3 to 12 lbs.
- Toy: 5 to 12 lbs.

Facilities/Tools Required:

- Python (JupyterLabs)
- Python (Google Colab)
- Kaggle
- Visual Studio (Just for looks.)

This project will be pure programing with no use of a training board.

The code will use certain libraries to manipulate images, search through folders, other math parameters, and CNN and other Pretrained Networks identifiers. Set parameters in the template would be: Mounting Google Drive with set paths, directory error, image dimensions, batch sizes, identify the number of weight classes which is 4, augment the data, train the data which sets up the epochs, evaluate the model, and then identify the test image.

Images 1 through 5 will identify each CNN and other Pretrained Networks codes used.

Table 1 showcases the folder architecture.

Table 2 showcases All 120 dog breeds in their weight class based on how heavy they can get.

```
# Create a CNN model
model = Sequential()
model.add(Conv2D(32, (3, 3), input_shape=(img_width, img_height, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Conv2D(128, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(256, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(num_classes, activation='softmax'))
```

Image 1: The set up for a CNN.

```
# Load pre-trained VGG16 model
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(img_width, img_height, 3))

# Freeze convolutional base
base_model.trainable = False

# Create the model
model = Sequential()
model.add(base_model)
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(num_classes, activation='softmax'))
```

Image 2: The set-up for a VGG16.

```
# Define AlexNet model
def create_alexnet_model(input_shape, num_classes):
    model = Sequential()

    # Layer 1
    model.add(Conv2D(96, (11, 11), strides=(4, 4), activation='relu', input_shape=input_shape))
    model.add(MaxPooling2D(pool_size=(3, 3), strides=(2, 2)))

    # Layer 2
    model.add(Conv2D(256, (5, 5), activation='relu'))
    model.add(MaxPooling2D(pool_size=(3, 3), strides=(2, 2)))

    # Layer 3
    model.add(Conv2D(384, (3, 3), activation='relu'))

    # Layer 4
    model.add(Conv2D(384, (3, 3), activation='relu'))

    # Layer 5
    model.add(Conv2D(256, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(3, 3), strides=(2, 2)))

    # Flatten the output and add dense layers
    model.add(Flatten())
    model.add(Dense(4096, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(4096, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(num_classes, activation='softmax'))

    return model
```

Image 3: The set-up for AlexNet.

```
# Load pre-trained InceptionV3 model
base_model = InceptionV3(weights='imagenet', include_top=False, input_shape=(img_width, img_height, 3))

# Add custom layers on top of InceptionV3
model = Sequential()
model.add(base_model)
model.add(GlobalAveragePooling2D())
model.add(Dense(256, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(num_classes, activation='softmax'))
```

Image 4: The set-up for InceptionV3 or GoogLeNet.

```
# Load pre-trained DenseNet201 model
base_model = DenseNet201(weights='imagenet', include_top=False, input_shape=(img_width, img_height, 3))

# Add custom layers on top of DenseNet201
model = Sequential()
model.add(base_model)
model.add(GlobalAveragePooling2D())
model.add(Dense(256, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(num_classes, activation='softmax'))

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.0001), loss='categorical_crossentropy', metrics=['accuracy'])
```

Image 5: The set-up for DenseNet201.

B. Block Diagrams

```

From Google Drive
|-- MyDrive
|   |-- ColabNotebooks
|   |   |-- Project
|   |   |   |-- Dataset
|   |   |   |   |-- Test
|   |   |   |   |   |-- Giant or Extra-Large: 100lbs or more
|   |   |   |   |   |   |-- African_Hunting_Dog
|   |   |   |   |   |   ...
|   |   |   |   |   |   |-- Large Dog Breeds: 59 to 99 lbs
|   |   |   |   |   |   |   |-- Basset_Hound
|   |   |   |   |   |   |   ...
|   |   |   |   |   |   |   |-- Medium Breeds: 24 to 57 lbs
|   |   |   |   |   |   |   |   |-- American_Staffordshire_Terrier
|   |   |   |   |   |   |   |   ...
|   |   |   |   |   |   |   |   |-- Small Dog Breeds: 2 to 22 lbs
|   |   |   |   |   |   |   |   |   |-- Miniature Breeds (3 to 12 lbs)*:
|   |   |   |   |   |   |   |   |   |   |-- Appenzeller
|   |   |   |   |   |   |   |   |   |   ...
|   |   |   |   |   |   |   |   |   |   |-- Teacup Breeds (4 lbs or less)
|   |   |   |   |   |   |   |   |   |   |   |-- Australian_Terrier
|   |   |   |   |   |   |   |   |   |   |   ...
|   |   |   |   |   |   |   |   |   |   |   |-- Toy Breeds (5 to 12 lbs
|   |   |   |   |   |   |   |   |   |   |   |   |-- Beagle
|   |   |   |   |   |   |   |   |   |   |   |   ...
|   |   |   |   |   |   |   |   |   |   |   |   |-- Train
|   |   |   |   |   |   |   |   |   |   |   |   ...
|   |   |   |   |   |   |   |   |   |   |   |   |-- Validation
|   |   |   |   |   |   |   |   |   |   |   |   ...
*** Note: The Train and Validation folders will repeat the same structure as the Test Folder.

```

Table 1: Folder Structure

```

# Define the dog breeds and their weight categories

dog_breeds = {

    "Giant or Extra-Large Dog Breeds (100 lbs. or more)": ["African_Hunting_Dog", "Great_Dane", "Great_Pyrenees", "Irish_Wolfhound", "Newfoundland", "Old_English_Sheepdog", "Saint_Bernard", "Tibetan_Mastiff"],

    "Large Dog Breeds (59 to 99 lbs.): [
    "Basset_Hound", "Bernese_Mountain_Dog", "Black_and_Tan_Coonhound", "Bloodhound", "Bluetick", "Borzoi", "Bouvier_Des_Flandres", "Brabancon_Griffon", "Brittany_Spaniel", "Cairn", "Cardigan", "Chesapeake_Bay_Retriever", "Clumber_Spaniel", "Dandie_Dinmont", "Dhole", "English_Foxhound", "English_Springer_Spaniel", "EntleBucher", "Flat_Coated_Retriever", "German_Shepherd", "Golden_Retriever", "Gordon_Setter", "Greater_Swiss_Mountain_Dog", "Groenendael", "Ibizan_Hound", "Irish_Setter", "Irish_Water_Spaniel", "Italian_Greyhound", "Japanese_Spaniel", "Labrador_Retriever", "Leonberg", "Malamute", "Mexican_Hairless", "Otterhound", "Papillon", "Redbone", "Rhodesian_Ridgeback", "Rottweiler", "Samoyed", "Schipperke", "Scotch_Terrier", "Scottish_Deerhound", "Sealyham_Terrier", "Shetland_Sheepdog", "Siberian_Husky", "Staffordshire_Bull_Terrier", "Standard_Poodle", "Standard_Schnauzer", "Sussex_Spaniel", "Vizsla", "Walker_Hound", "Weimaraner", "Wire_Gaired_Fox_Terrier", "Wire_Haired_Fox_Terrier"],

    "Medium Dog Breeds (24 to 57 lbs.): [
    "American_Staffordshire_Terrier", "Afghan_Hound", "Airedale", "Appenzeller", "Blenheim_Spaniel", "Boxer", "Briard", "Bull_Mastiff", "Chesapeake_Bay_Retriever", "Chow_Chow", "Clumber_Spaniel", "Collie", "Curly_Coated_Retriever", "Doberman", "Dingo", "English_Setter", "Eskimo_Dog", "Flat_Coated_Retriever", "German_Short_Haired_Pointer", "Giant_Schnauzer", "Irish_Terrier", "Irish_Water_Spaniel", "Italian_Greyhound", "Japanese_Spaniel", "Keeshond", "Kelpie", "Kerry_Blue_Terrier", "Komondor", "Kuvasz", "Labrador_Retriever", "Lakeland_Terrier", "Leonberg", "Lhasa", "Malamute", "Malinois", "Mexican_Hairless", "Miniature_Poodle", "Norfolk_Terrier", "Norwegian_Elkhound", "Otterhound", "Papillon", "Pekinese", "Pembroke", "Redbone", "Rhodesian_Ridgeback", "Rottweiler", "Saluki", "Samoyed", "Schipperke", "Scotch_Terrier", "Scottish_Deerhound", "Sealyham_Terrier", "Shetland_Sheepdog", "Shih_Tzu", "Siberian_Husky", "Silky_Terrier", "Soft_Coated_Wheaten_Terrier", "Sussex_Spaniel", "Tibetan_Mastiff", "Walker_Hound", "Weimaraner", "Welsh_Springer_Spaniel", "West_Highland_White_Terrier", "Whippet", "Wire_Haired_Fox_Terrier"],

    "Small Dog Breeds (12 to 22 lbs.): ["Teacup Breeds (4 lbs or less)", "Toy Breeds (5 to 12 lbs)", "Miniature Breeds (3 to 12 lbs)"],

    "Teacup Breeds (4 lbs or less)": ["Australian_Terrier", "Basenji", "Border_Terrier", "Brussels_Griffon", "Chihuahua", "Italian_Greyhound", "Maltese", "Miniature_Pinscher", "Papillon", "Pekingese", "Pomeranian", "Yorkshire_Terrier"],

    "Miniature Breeds (3 to 12 lbs)": ["Appenzeller", "Affenpinscher", "Bedlington_Terrier", "Bichon_Frise", "Cairn_Terrier", "Dachshund", "Dandie_Dinmont", "Norfolk_Terrier", "Norwich_Terrier", "Pug", "Scottish_Terrier", "Shetland_Sheepdog", "Silky_Terrier", "Toy_Poodle", "Toy_Terrier", "Welsh_Springer_Spaniel", "West_Highland_White_Terrier"],

    "Toy Breeds (5 to 12 lbs.): ["Beagle", "Border_Collie", "Boston_Terrier", "Bulldog", "Cocker_Spaniel", "French_Bulldog", "Keeshond", "Kerry_Blue_Terrier", "Miniature_Schnauzer", "Shih_Tzu", "Sussex_Spaniel", "Tibetan_Terrier"]

}

```

Table 2: All 120 dog breeds in their weight class as heavy as they can get.

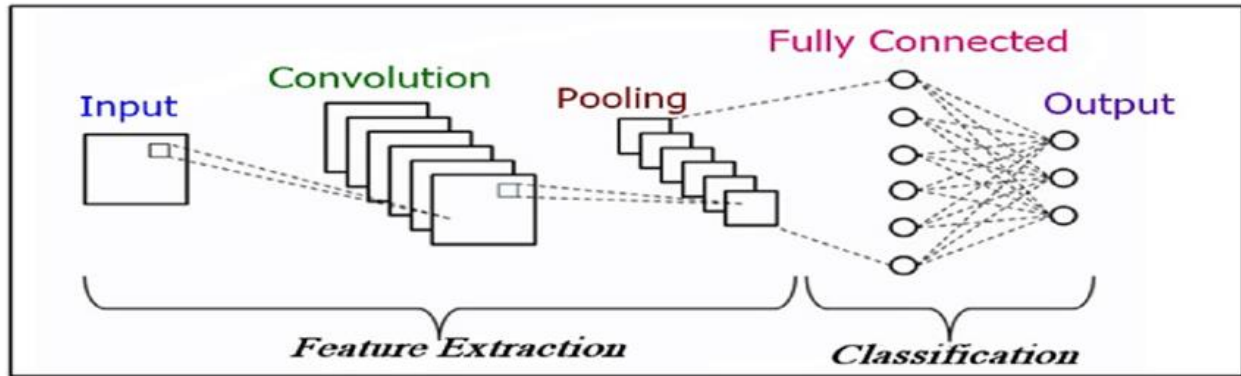


Diagram 1: CNN Block Diagram

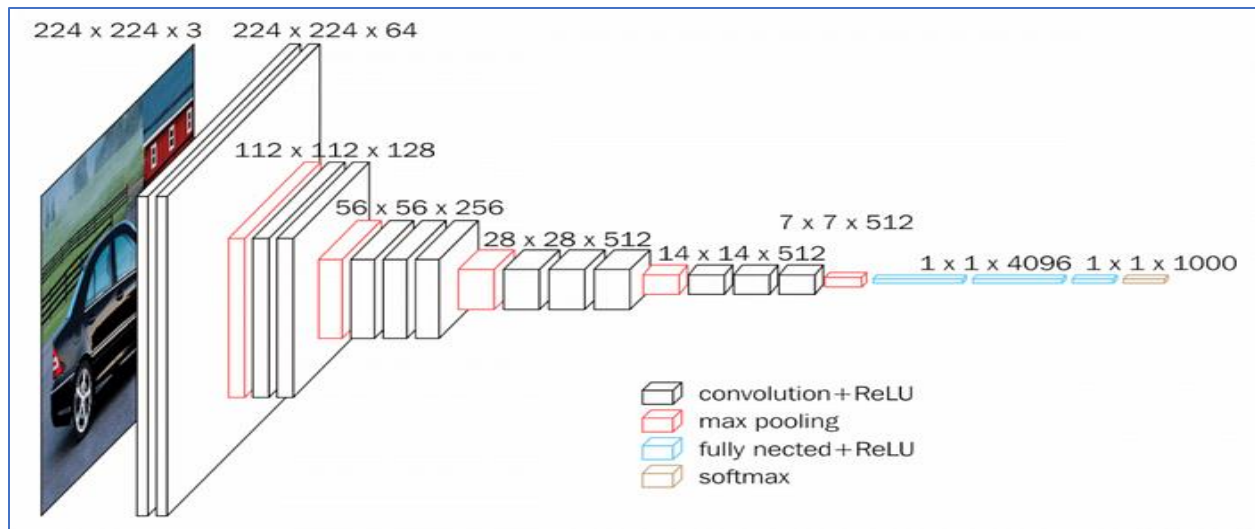


Diagram 2: VGG16 Block Diagram

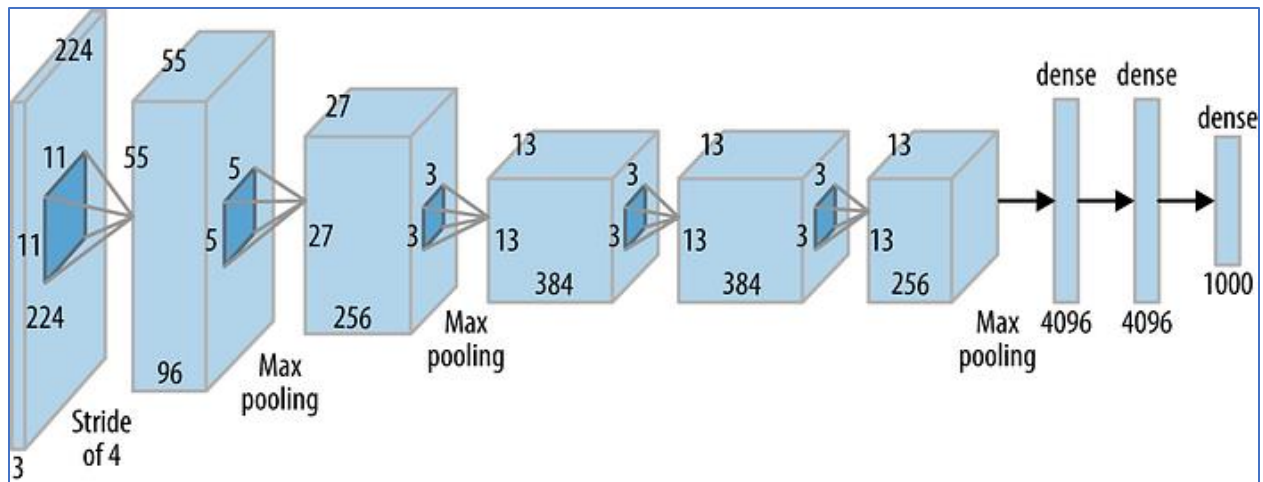


Diagram 3: Alexnet Block Diagram

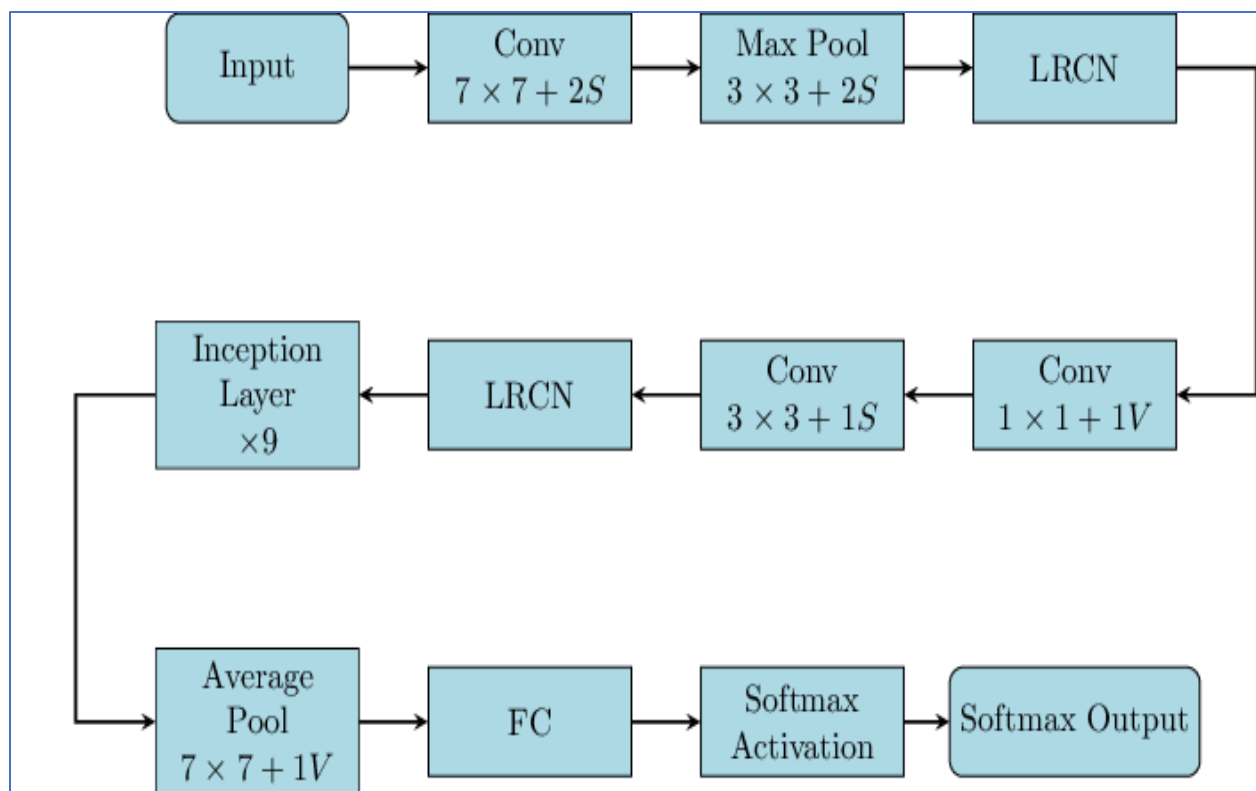


Diagram 4: GoogLeNet Block Diagram

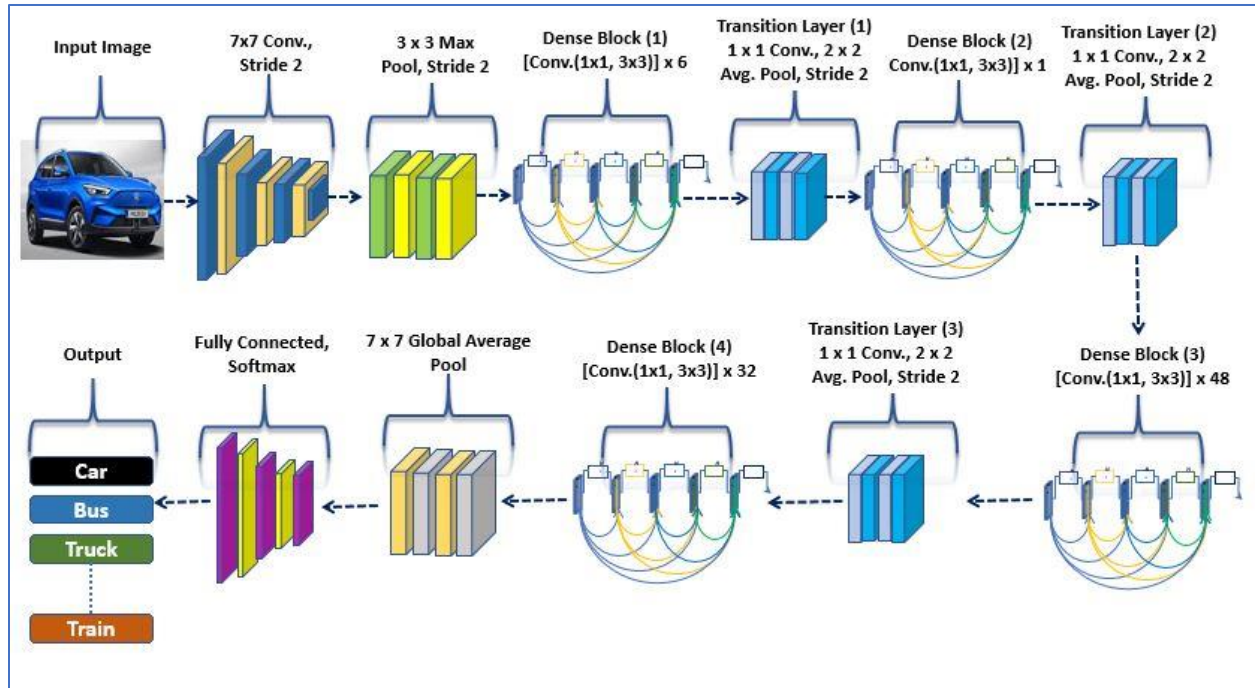


Diagram 5: DenseNet201 Block Diagram

C. Theory:

The computational sequence of a convolutional layer flows as the following info enters the dendrite, the data gets into a convolution, then gets an activation, follows through a normalization, floats into a pooling, and results from the axon.

Each of the multi-layer neural networks architectures have the following purposes:

- CNN is used for finding patterns in images to recognize objects, classes, and categories.
- VVG16 is used for classifying images into 1000 object categories.
- AlexNet is used for image recognition and classification tasks.
- GoogLeNet is used for image classification and recognition.
- DenseNet201 is used for picture classification, object identification, and semantic segmentation.

As seen in Diagrams 1 thru 5, each details the basic follow of start to finish.

Pretrained Networks on ImageNet (Matlab)

Network	Depth	Size (MB)	Parameters (M)	Input Image Size
SqueezeNet	18	5.2	1.24	227x227
GoogleNet	22	27	7	224x224
MobileNet v2	53	13	3.5	224x224
DenseNet 201	201	77	20	224x224
Inception v3	48	89	23.9	299x299
ShuffleNet	50	5.4	1.4	224x224
Resnet50	50	96	25.6	224x224
EfficientNet b0	82	20	5.3	224x224
AlexNet	8	227	61	227x227
VGG16	16	515	138	224x224

2023

EE/SJSU

Image 6: A List of Pretrained Networks.

Image 6 provides what Pretrained Networks can be selected. I selected VGG16, AlexNet, GoogLeNet, and DenseNet201 to perform and compare.

VGG16 was selected due to being the biggest in size.

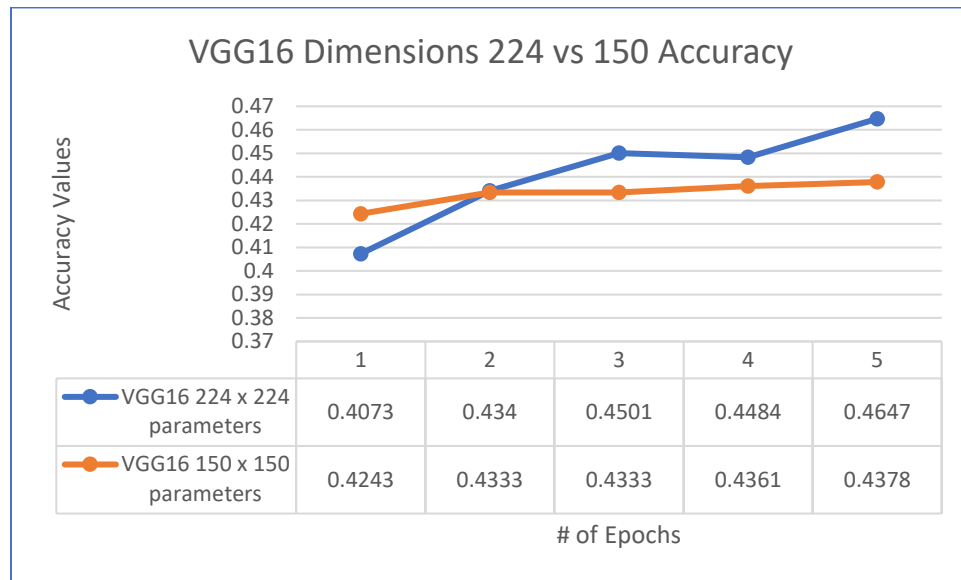
AlexNet was selected due to being the first pretrained network created by a machine.

GoogLeNet was selected due to name's sake.

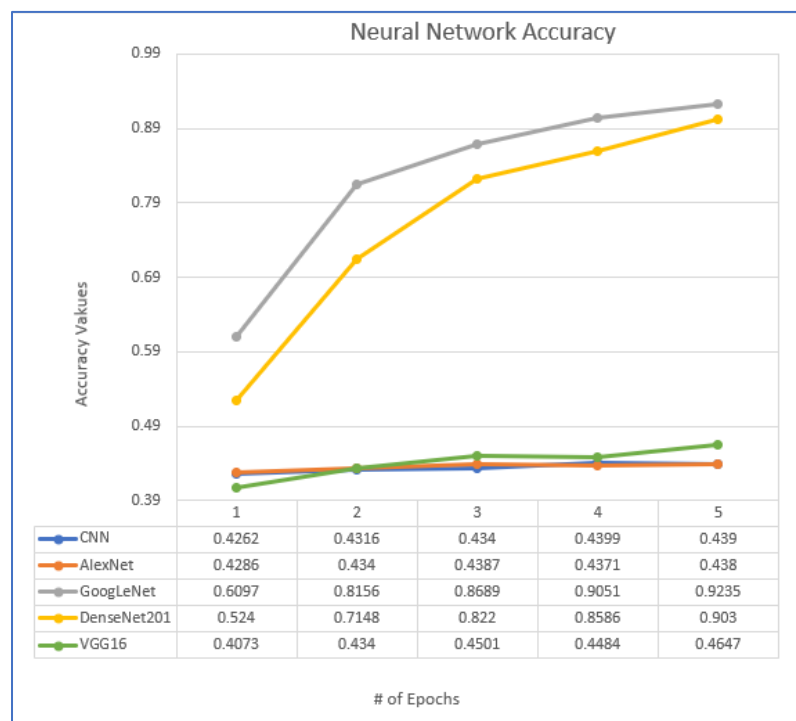
DenseNet201 was selected due to having the most depth.

3. Simulation/Verification

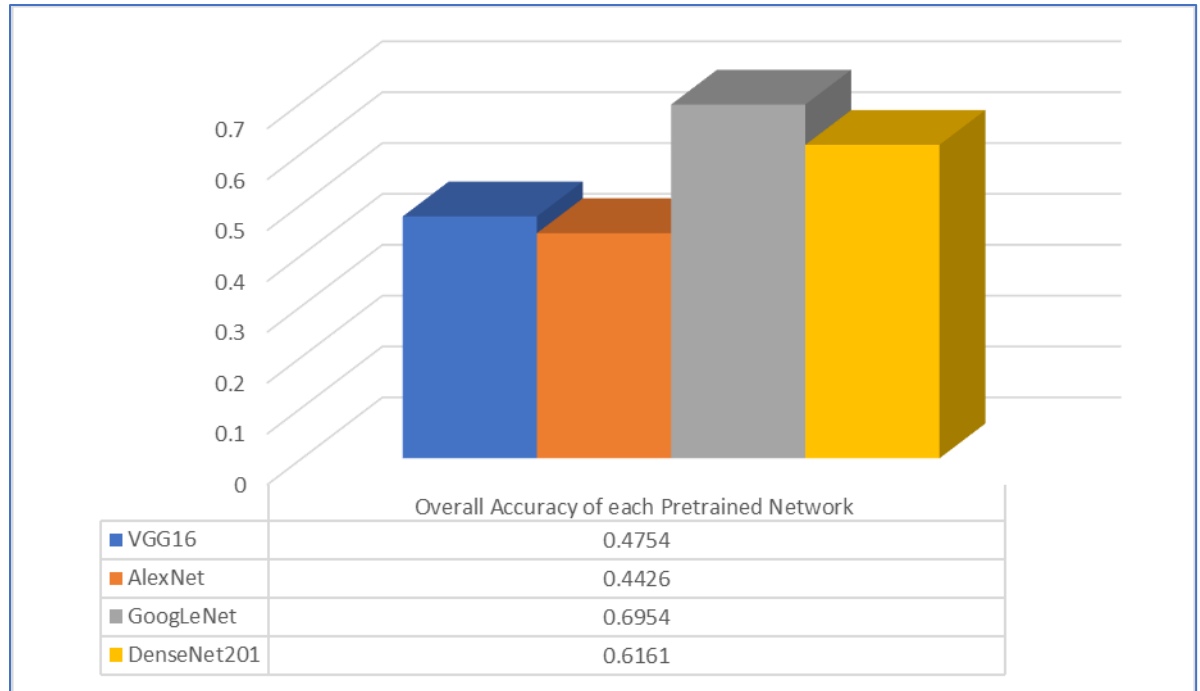
A. Verification Results



Graph 1: VGG16 Dimensions 224 vs 150 Accuracy



Graph 2: Neural Network Accuracy



Graph 3: Overall Accuracy of Each Pretrained Network

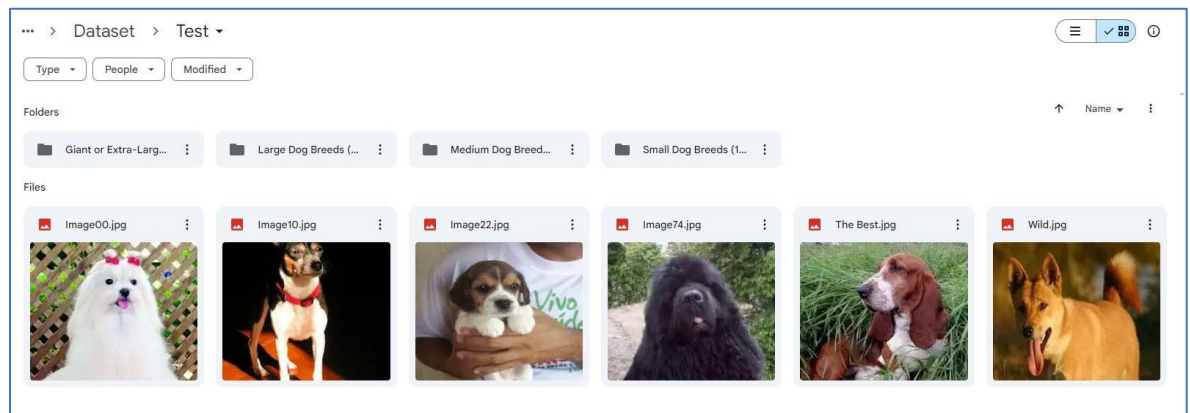


Image 7: The Test Images

CNN 1/1 [=====] - 0s 124ms/step Image: Image00.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 32ms/step Image: Image10.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 33ms/step Image: Image74.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 33ms/step Image: The Best.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 33ms/step Image: Image22.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 32ms/step Image: Wild.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.)	
VGG16 1/1 [=====] - 0s 211ms/step Image: Image00.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 73ms/step Image: Image10.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 97ms/step Image: Image74.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 86ms/step Image: The Best.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 74ms/step Image: Image22.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 75ms/step Image: Wild.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.)	GoLeNet 1/1 [=====] - 1s 1s/step Image: Wild.jpg, Predicted Class: Medium Dog Breeds (24 to 57 lbs.) 1/1 [=====] - 0s 25ms/step Image: The Best.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 25ms/step Image: Image10.jpg, Predicted Class: Small Dog Breeds (12 to 22 lbs.) 1/1 [=====] - 0s 23ms/step Image: Image22.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 23ms/step Image: Image74.jpg, Predicted Class: Giant or Extra-Large Dog Breeds (100 lbs. or more) 1/1 [=====] - 0s 25ms/step Image: Image00.jpg, Predicted Class: Small Dog Breeds (12 to 22 lbs.)
AlexNet 1/1 [=====] - 0s 321ms/step Image: Wild.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 19ms/step Image: The Best.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 17ms/step Image: Image10.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 18ms/step Image: Image22.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 24ms/step Image: Image74.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 18ms/step Image: Image00.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.)	DenseNet201 1/1 [=====] - 6s 6s/step Image: Image00.jpg, Predicted Class: Small Dog Breeds (12 to 22 lbs.) 1/1 [=====] - 0s 62ms/step Image: Image10.jpg, Predicted Class: Small Dog Breeds (12 to 22 lbs.) 1/1 [=====] - 0s 80ms/step Image: Image74.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 70ms/step Image: The Best.jpg, Predicted Class: Large Dog Breeds (59 to 99 lbs.) 1/1 [=====] - 0s 35ms/step Image: Image22.jpg, Predicted Class: Small Dog Breeds (12 to 22 lbs.) 1/1 [=====] - 0s 34ms/step Image: Wild.jpg, Predicted Class: Medium Dog Breeds (24 to 57 lbs.)

Table 3: Neural Network Test Image Match-up Results

B. Discussion

When selecting what neural network to use a test of parameters must be used. In class and through homework the batch size (32), epochs (5), and learning rate (0.0001) have been tested enough to know what numbers to use. The higher the epoch the more each neural network will gain an accuracy nearing 100%, but that could take 10 to 30 epochs. I did not have the luxury of time and set the epochs to 5 as a common base line. Along with the topic of reducing speed, the initial number of images from the dataset was 20,669 images and I reduced it to 6,055 for all 120 breeds of canines. One aspect that I have not tested is changing the dimensions of each neural network. In Graph 1, VGG16 Dimensions 224 vs 150 Accuracy, it shows that keeping the dimensions in the defined max range will better yield higher accuracy ratings. Both Graph 2 and Graph 3 relays how accurate each neural network's epoch was. Image 7, contains the test images each neural network will match what it thinks what weight class each image belongs to.

These are the test images and what weight class that they should belong to:

- Image00 = Small Dog Breeds: 2 to 22 lbs.
- Image10 = Small Dog Breeds: 2 to 22 lbs.
- Image22 = Small Dog Breeds: 2 to 22 lbs.
- Image74 = Giant or Extra-Large: 100lbs. or more
- The Best = Large Dog Breeds: 59 to 99 lbs.

- Wild = Medium Breeds: 24 to 57 lbs.

Time to simplify Table 3 results with Following list.

These Neural Networks have some amounts correct out of 6 and an explanation on how it errored:

- CNN = 1, Only guessed Large Dog Breeds: 59 to 99 lbs.
- VGG16 = 1, Only guessed Large Dog Breeds: 59 to 99 lbs.
- AlexNet = 1, Only guessed Large Dog Breeds: 59 to 99 lbs.
- GoogLeNet = 5, Called Image22 Large Dog Breeds: 59 to 99 lbs.
- DenseNet201 = 5, Called Image74 Large Dog Breeds: 59 to 99 lbs.

Not perfect, but the yields shown makes sense with the results.

4. Conclusion

A. Project Summary

I took a dog gone situation of identifying man's best friend by use of neural networks. Reduced the dataset to a workable means out of the 120 breeds. Sorted the breeds into their weight classes. Test, train, and validate the dataset. Created a program template to convolute the training dataset and then identify the sample test images while being able to swap in either a neural network.

B. Goal Fulfillment

I found that found that GoogLeNet and DenseNet201 are the best trained networks to use.

GoogLeNet yields the highest overall accuracy at 69.54%, on the fifth epoch it resulted a 92.35%, and matched 5/6 test images correctly.

DenseNet201 yields the highest overall accuracy at 61.61%, on the fifth epoch it resulted a 90.30%, and matched 5/6 test images correctly.

I am satisfied with main feature of this project being completed.

C. Future Work

If more time was permitted, I would have this as an app for any smart phone.

These are some of the features I had to drop:

- Produce an error to occur if an image not of a dog image is tested
- With smaller breeds, further specify Miniature, Teacup, and Toy subclasses.
- Identify each breed of dog.

A side from time, the other reason these features were dropped was due to constant troubleshooting error.

5. References

- Jesucristo (2019). Title of the notebook: Introducing DCGAN Dogs Images. Kaggle. <https://www.kaggle.com/code/jesucristo/introducing-dcgan-dogs-images>
- ChasingTails Team (November 03, 2021). Title of the webpage: Dog Sizes: A Guide to Different Dog Breeds and Their Sizes. Chasing Tails. URL: <https://chasingtails.store/blogs/tips/dog-sizes>
- Ghosh, Soumadip & Banerjee, Suharta & Das, Supantha & Hazra, Arnab & Mallik, Saurav & Zhao, Zhongming & Mukherji, Ayan. (2022). Evaluation and Optimization of Biomedical Image-Based Deep Convolutional Neural Network Model for COVID-19 Status Classification. Applied Sciences. 12. 10787. 10.3390/app122110787. (CNN Block Diagram)
- Author(s): Aqeel Anwar (Jun 7, 2019). Title of the webpage: Difference between AlexNet, VGGNet, ResNet, and Inception. Chasing Tails. URL: <https://towardsdatascience.com/the-w3h-of-alexnet-vggnet-resnet-and-inception-7baaaecccc96> (AlexNet and VGG16 Block Diagram)
- Singh, Sanjay & Kishore, Anurag. (2015). Natural Language Image Descriptor. 10.1109/RAICS.2015.7488398. (GoogLeNet Block Diagram)
- Akhtar, Malik & Mahum, Rabbia & Shafique Butt, Faisal & Amin, Rashid & El-Sherbeeney, Ahmed & Lee, Seongkwan & Shaikh, Sarang. (2022). A Robust Framework for Object Detection in a Traffic Surveillance System. Electronics. 11. 3425. 10.3390/electronics11213425. (DenseNet201 Block Diagram)
- MathWorks. "CNN & Pretrained Networks - MathWorks." MathWorks, 2022, <https://www.mathworks.com/>. Accessed 10 March 2024. (Multiple pages on this site)